

Software Requirements Specification

A Mobile Application for Facility Management

Team

.

Members

Joy Bassem | 13003606 (Team
Lead)

Hana Khaled | 13002847

Malak Fawzy | 13007189

Maryam Basim | 13005731

Mohamed Adel | 13002519

Adam Sherbiny | 13002518

Tutorial Details

T4 | Hassan Abdelqader

1. Introduction

Campus environments require continuous monitoring and maintenance to ensure that facilities remain safe, clean, and functional for students and staff. However, issues such as broken furniture, flickering lights, plumbing problems, or cleanliness concerns can often remain unresolved when there is no structured reporting and tracking system.

CampusCare is a mobile application designed to improve the way campus maintenance issues are reported, managed, assigned, and resolved. The system provides a clear communication channel between community members, facility managers, workers, and system administrators. Instead of reporting issues informally, users can submit a digital ticket that includes a title, description, category, location, and image evidence.

The main purpose of CampusCare is to support faster and more organized maintenance management. Community members can report issues and track their progress. Facility Managers can view submitted issues, assign workers, update issue status, set priorities, and manage maintenance tasks. Workers can view their assigned issues, update progress, add comments, and upload completion photos. Admins can manage user accounts by activating or deactivating users.

The intended audience for this document includes software developers, project stakeholders, testers, system administrators, and university management. It also serves as a reference for future developers who may improve or expand the system.

The scope of CampusCare includes a mobile frontend built with React Native with Expo and a backend built with Node.js, Express.js, a hosted database using Supabase PostgreSQL, and cloud-based image storage using Cloudinary. The system supports user authentication, role-based access control, issue management, image uploading through Cloudinary, notifications, worker assignment, and user account management.

1.1 Product Vision & Scope

The vision of CampusCare is to provide a simple, organized, and efficient digital platform for managing campus maintenance issues. The system aims to replace informal reporting methods with a structured mobile application that allows issues to be submitted, tracked, assigned, and resolved through one centralized platform.

CampusCare improves communication between the university community and the Facility Management team. Community Members can submit maintenance tickets directly through the app, while Facility Managers can review, prioritize, assign, and manage these tickets through a dedicated dashboard. This ensures that maintenance problems are recorded properly and handled more efficiently.

Another objective of the system is to reduce response time for maintenance issues. Once an issue is submitted, the Facility Manager can review it, assign it to an appropriate worker, set its priority, and monitor progress. Workers can then update the task status, add work comments, and upload a completion photo once the issue is resolved.

The system also supports better record keeping. Since all users, issues, comments, notifications, categories, locations, priorities, statuses, and image URLs are stored in Supabase PostgreSQL, the university can maintain a structured history of maintenance activities. This data can support future reporting, analysis, and decision-making.

The current system focuses on four main user roles: Community Member, Facility Manager, Worker, and System Admin. Each role has specific permissions based on role-based access control.

1.2 Definitions and Acronyms

SRS: Software Requirements Specification

CampusCare: The mobile system used to report and manage campus maintenance issues

Issue / Ticket: A digital record representing a maintenance issue reported through the system

Community Member: A student or staff member who can submit maintenance tickets

Facility Manager: A user responsible for reviewing tickets and assigning workers

Worker: A user responsible for resolving assigned maintenance issues

Admin: A system administrator who manages users, roles, and locations

RBAC: Role-Based Access Control, a security method that restricts system access based on user roles

React Native: Framework used to build the mobile application

Expo: Development platform used to run and test the React Native mobile application

Node.js: Technology used to develop the backend server

Express.js: Backend framework used to create RESTful API routes

Supabase PostgreSQL: Hosted database service used to store system's structured data

Cloudinary: Cloud storage service used to store uploaded issue images and completion photos

2. User Stories

-Community Member User Stories

US-CM-01:

As a Community Member, I should be able to create an account so that I can access the application.

US-CM-02:

As a Community Member, I should be able to log in using my email and password so that I can use the system securely.

US-CM-03:

As a Community Member, I should be able to submit a maintenance issue so that the Facility Management team can review it.

US-CM-04:

As a Community Member, I should be able to add an issue title, description, category, location, and photo so that the problem is reported clearly.

US-CM-05:

As a Community Member, I should be able to upload an issue image using the device camera or gallery so that I can provide visual evidence.

US-CM-06:

As a Community Member, I should be able to view my submitted issues so that I can track their progress.

US-CM-07:

As a Community Member, I should be able to view the status of my issue, such as Pending, Assigned, In Progress, Resolved, or Closed.

US-CM-08:

As a Community Member, I should be able to view notifications related to my submitted issues so that I can stay informed about updates.

US-CM-09:

As a Community Member, I should be able to mark notifications as read so that I can organize my updates.

US-CM-10:

As a Community Member, I should be able to log out so that my account remains secure.

US-CM-11:

As a Community Member, I should be able to reset my password through a simplified password reset process if I forget my login details.

-Facility Manager User Stories

US-FM-01:

As a Facility Manager, I should be able to create an account so that I can access the management features of the application.

US-FM-02:

As a Facility Manager, I should be able to log in using my email and password so that I can manage submitted issues securely.

US-FM-03:

As a Facility Manager, I should be able to view all submitted issues so that I can monitor campus maintenance requests.

US-FM-04:

As a Facility Manager, I should be able to view issue details so that I can understand the reported problem.

US-FM-05:

As a Facility Manager, I should be able to assign an issue to a worker so that the issue can be handled by the correct person.

US-FM-06:

As a Facility Manager, I should be able to update the status of an issue so that the system reflects the current progress.

US-FM-07:

As a Facility Manager, I should be able to set the priority of an issue so that urgent problems can be handled first.

US-FM-08:

As a Facility Manager, I should be able to close an issue after it has been resolved.

US-FM-09:

As a Facility Manager, I should be able to search and filter issues so that I can manage maintenance tasks efficiently.

US-FM-10:

As a Facility Manager, I should be able to delete submitted issues when necessary.

US-FM-11:

As a Facility Manager, I should be able to activate or deactivate worker accounts so that only valid workers can access the system.

US-FM-12:

As a Facility Manager, I should be able to view notifications when important issue updates occur.

US-FM-13:

As a Facility Manager, I should be able to mark notifications as read so that I can manage updates clearly.

US-FM-14:

As a Facility Manager, I should be able to log out so that my account remains secure.

US-FM-15:

As a Facility Manager, I should be able to reset my password through a simplified password reset process.

-Worker User Stories

US-W-01:

As a Worker, I should be able to create an account so that I can access assigned maintenance tasks.

US-W-02:

As a Worker, I should be able to log in using my email and password so that I can view my assigned issues.

US-W-03:

As a Worker, I should be able to view issues assigned to me so that I know which tasks I need to complete.

US-W-04:

As a Worker, I should be able to view full issue details, including title, description, category, location, status, priority, and image.

US-W-05:

As a Worker, I should be able to mark an assigned issue as In Progress so that the Facility Manager and Community Member know that work has started.

US-W-06:

As a Worker, I should be able to add comments to an issue so that I can provide updates about the maintenance work.

US-W-07:

As a Worker, I should be able to upload a completion photo so that the Facility Manager can verify the completed work.

US-W-08:

As a Worker, I should be able to mark an issue as resolved after uploading a completion photo.

US-W-09:

As a Worker, I should receive assignment updates through the system when an issue is assigned to me.

US-W-10:

As a Worker, I should be able to log out so that my account remains secure.

US-W-11:

As a Worker, I should be able to reset my password through a simplified password reset process.

-System Admin User Stories

US-SA-01:

As a System Admin, I should be able to log in so that I can access system administration features.

US-SA-02:

As a System Admin, I should be able to view all registered users so that I can monitor system accounts.

US-SA-03:

As a System Admin, I should be able to activate or deactivate user accounts so that I can control system access.

US-SA-04:

As a System Admin, I should be able to log out so that my account remains secure.

2.1 Functional Requirements

-Community Member Functional Requirements

FR-CM-01: Account Creation

The system shall allow Community Members to create an account by providing the required registration details.

FR-CM-02: User Authentication

The system shall allow Community Members to log in using a valid email and password.

FR-CM-03: Simplified Password Reset

The system shall allow Community Members to reset forgotten passwords using a simplified password reset process. A future version can improve this using secure email reset tokens.

FR-CM-04: Issue Submission

The system shall allow Community Members to submit maintenance issues.

FR-CM-05: Issue Details

The system shall allow Community Members to provide an issue title, description, category, location, and image.

FR-CM-06: Image Upload

The system shall allow Community Members to upload issue images using the device camera or gallery.

FR-CM-07: View Submitted Issues

The system shall allow Community Members to view a list of issues they submitted.

FR-CM-08: View Issue Status

The system shall display the current status of each issue to the Community Member.

FR-CM-09: View Issue Details

The system shall allow Community Members to open an issue and view its full details, including issue information, status updates, and worker comments where available.

FR-CM-10: Notifications

The system shall allow Community Members to view notifications related to issue updates.

FR-CM-11: Notification Management

The system shall allow Community Members to mark notifications as read.

FR-CM-12: Logout

The system shall allow Community Members to log out of the application.

-Facility Manager Functional Requirements

FR-FM-01: Account Creation

The system shall allow Facility Managers to create an account.

FR-FM-02: User Authentication

The system shall allow Facility Managers to log in using a valid email and password.

FR-FM-03: View All Issues

The system shall allow Facility Managers to view all submitted maintenance issues.

FR-FM-04: View Issue Details

The system shall allow Facility Managers to view full issue details, including title, description, category, location, status, priority, assigned worker, submitted image, and submission date.

FR-FM-05: Assign Issue to Worker

The system shall allow Facility Managers to assign issues to workers.

FR-FM-06: Update Issue Status

The system shall allow Facility Managers to update the status of an issue.

FR-FM-07: Set Issue Priority

The system shall allow Facility Managers to set issue priority levels such as Low, Medium, High, or Urgent.

FR-FM-08: Close Issue

The system shall allow Facility Managers to close an issue after it has been resolved.

FR-FM-09: Search Issues

The system shall allow Facility Managers to search submitted issues using keywords.

FR-FM-10: Filter Issues

The system shall allow Facility Managers to filter issues using available frontend filters such as status, category, and date. The backend also supports additional filtering options such as location and priority.

FR-FM-11: Delete Issue

The system shall allow Facility Managers to delete submitted issues when necessary.

FR-FM-12: Worker Account Management

The system shall allow Facility Managers to view workers and activate or deactivate worker accounts.

FR-FM-13: Manager Notifications

The system shall notify Facility Managers when important issue updates occur, such as worker comments or completed tasks.

FR-FM-14: Notification Management

The system shall allow Facility Managers to view notifications and mark them as read.

FR-FM-15: Logout

The system shall allow Facility Managers to log out of the application.

FR-FM-16: Simplified Password Reset

The system shall allow Facility Managers to reset forgotten passwords using a simplified password reset process.

-Worker Functional Requirements

FR-W-01: Account Creation

The system shall allow Workers to create an account.

FR-W-02: User Authentication

The system shall allow Workers to log in using a valid email and password.

FR-W-03: View Assigned Issues

The system shall allow Workers to view maintenance issues assigned to them.

FR-W-04: View Issue Details

The system shall allow Workers to view full details of assigned issues, including title, description, category, location, status, priority, submitted image, and submission date.

FR-W-05: Update Status to In Progress

The system shall allow Workers to mark an assigned issue as In Progress.

FR-W-06: Add Comments

The system shall allow Workers to add comments to an assigned issue to explain work progress or updates.

FR-W-07: Upload Completion Photo

The system shall allow Workers to upload a completion photo after finishing the task.

FR-W-08: Resolve Issue

The system shall allow Workers to mark an issue as resolved after uploading a completion photo.

FR-W-09: Assignment Updates

The system shall create assignment updates when an issue is assigned to a worker.

FR-W-10: Logout

The system shall allow Workers to log out of the application.

FR-W-11: Simplified Password Reset

The system shall allow Workers to reset forgotten passwords using a simplified password reset process.

-System Admin Functional Requirements

FR-SA-01: Admin Authentication

The system shall allow System Admins to log in using valid credentials.

FR-SA-02: View Registered Users

The system shall allow System Admins to view all registered users.

FR-SA-03: Activate User Accounts

The system shall allow System Admins to activate user accounts.

FR-SA-04: Deactivate User Accounts

The system shall allow System Admins to deactivate user accounts.

FR-SA-05: Logout

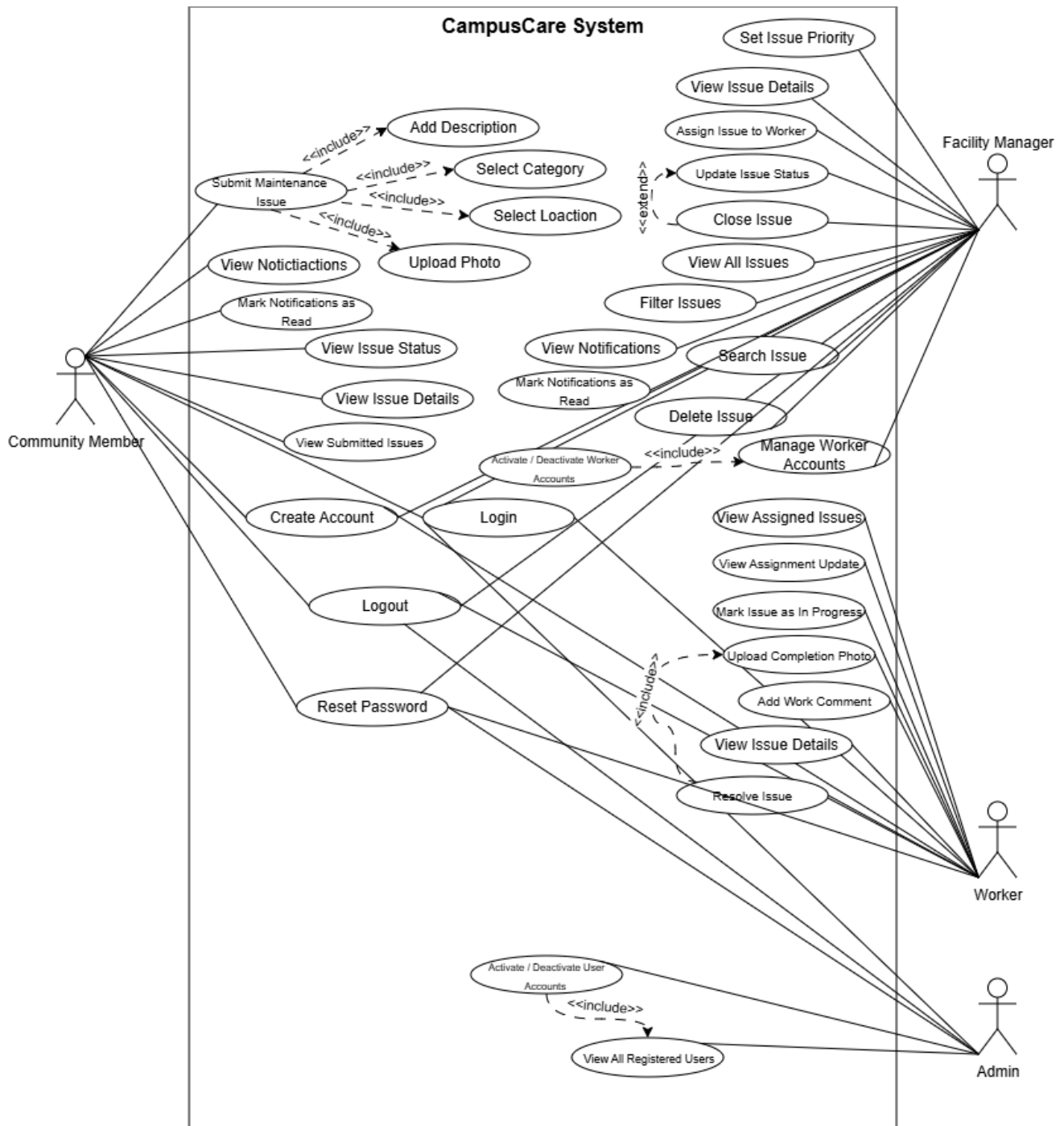
The system shall allow System Admins to log out of the application.

2.2 Product Backlog

Backlog ID	User Story ID	Priority
Core System Features		
PB-01	US-CM-01	High
PB-02	US-CM-02	High
PB-03	US-CM-03	High
PB-04	US-CM-04	High
PB-05	US-CM-05	High
PB-06	US-CM-06	High
PB-07	US-CM-07	High
PB-08	US-FM-01	High
PB-09	US-FM-02	High
PB-10	US-W-01	High
PB-11	US-W-02	High
Issue Management Features		
PB-12	US-FM-03	High
PB-13	US-FM-04	High
PB-14	US-FM-05	High
PB-15	US-FM-06	High
PB-16	US-FM-07	High
PB-17	US-FM-08	High
PB-18	US-FM-09	Medium
PB-19	US-FM-10	Medium
PB-20	US-FM-11	Medium

PB-21	US-W-03	High
PB-22	US-W-04	High
PB-23	US-W-05	High
PB-24	US-W-06	Medium
PB-25	US-W-07	Medium
PB-26	US-W-08	Medium
Authentication & Security		
PB-27	US-CM-10	Medium
PB-28	US-CM-11	Medium
PB-29	US-FM-14	Medium
PB-30	US-FM-15	Medium
PB-31	US-W-10	Medium
PB-32	US-W-11	Medium
Notifications		
PB-33	US-CM-08	Medium
PB-34	US-CM-09	Medium
PB-35	US-FM-12	Medium
PB-36	US-FM-13	Medium
PB-37	US-W-09	Medium
System Administration		
PB-38	US-SA-01	Low
PB-39	US-SA-02	Low
PB-40	US-SA-03	Low
PB-41	US-SA-04	Low

2.3 UML Use-Case Diagram



3. Non-Functional Requirements

- **Response Time:** The system should provide acceptable response time for normal actions such as login, issue submission, viewing issues, assigning workers, updating statuses, and uploading images.
- **Concurrent Users:** The system is designed using a client-server architecture that can support multiple users, but load testing is required to confirm support for 500 simultaneous users.

- **API Security:** JWT authentication, role-based access control, and input validation are implemented to protect the system. Rate limiting can be added in a future production version.
- **Backup & Recovery:** Supabase PostgreSQL stores the system data, but automated backups and disaster recovery should be configured through the hosting environment in a future production deployment.
- **Usability:** The system provides an intuitive mobile interface that allows users to submit maintenance tickets by entering issue details, selecting category and location, and uploading a photo.
- **Reliability:** The system stores uploaded images using Cloudinary and saves image URLs in Supabase PostgreSQL. Further testing is needed to confirm image upload reliability under unstable network conditions.

4. Constraints

The mobile application must be developed using React Native with Expo, while the backend must be implemented using Node.js and Express.js with RESTful APIs. The database must use Supabase PostgreSQL to store users, issues, comments, notifications, categories, locations, priorities, statuses, and uploaded image URLs. Image files uploaded with maintenance tickets must be stored using Cloudinary as a cloud-based storage service. The system must support mobile devices and requires stable internet connectivity for login, submitting tickets, uploading images, viewing updates, and accessing notifications. Security measures such as JWT authentication and role-based access control must be implemented to restrict access based on user roles.

5. Technology Stack

- **Frontend:** The CampusCare mobile application was developed using React Native with Expo to support cross-platform mobile development.
- **Backend:** The backend system was implemented using Node.js and Express.js to provide RESTful API services and business logic handling.
- **Database:** Supabase PostgreSQL was used as the hosted relational database for storing users, issues, comments, notifications, categories, locations, priorities, statuses, worker assignments, and uploaded image URLs.
- **Authentication & Authorization:** JWT authentication and Role-Based Access Control were implemented to secure the application and manage user permissions based on user roles.
- **Image Handling:** Multer and Cloudinary were integrated for image upload and cloud storage. Multer handles uploaded image files in the backend, while Cloudinary stores issue photos and worker completion photos.
- **Navigation & Storage:** React Navigation was used for screen navigation, while AsyncStorage was used for local session persistence, including storing the authentication token and logged-in user data.

6. Future Scope of the Project

- **Predictive Maintenance:** Analyze historical ticket data to predict recurring maintenance issues and support preventive maintenance planning.
- **Gamification:** Introduce points or badges for users who report valid issues and contribute to campus improvement.
- **Real-Time Ticket Tracking:** Add real-time progress tracking using technologies such as WebSockets or live database updates so users can immediately see changes.
- **Advanced Notifications:** Add push notifications, email alerts, or SMS alerts for ticket updates, assignments, and completed tasks.
- **Management Dashboard:** Develop a web dashboard for Facility Managers and university administrators to monitor system activity, issue trends, worker performance, and maintenance statistics.
- **AI Image Classification:** Use AI to automatically suggest issue categories based on uploaded images.